

۱. سناریو، اهداف و جامعه هدف

سناریو:

رستوران "سروش" با ۲۰ میز، ۵۰ آیتم منو و ۸ پرسنل، نیاز به سیستمی دارد که:

- ثبت سفارش با تبلت توسط گارسون
- ارسال مستقیم سفارش به آشپزخانه
- مدیریت موجودی انبار
- محاسبه خودکار صورت حساب با تخفیف‌های روزانه
- تحلیل فروش روزانه

اهداف:

- کاهش ۶۰٪ زمان انتظار مشتری
- کاهش ۴۰٪ خطاهای ثبت سفارش
- افزایش ۲۵٪ سود از طریق مدیریت بهینه موجودی
- ارائه داشبورد لحظه‌ای به مدیر

جامعه هدف:

مشتریان، گارسون‌ها، آشپزها، مدیر رستوران، انباردار، حسابدار

۶. معماری پیشنهادی و علت آن

معماری انتخاب شده: *Layered Architecture* (لایه‌های ۴ گانه)

لایه‌ها:

1. *Presentation (UI)*: اپلیکیشن تبلت گارسون، کیوسک مشتری، داشبورد مدیریت
2. *Business Logic (BLL)*: مدیریت سفارش، محاسبه تخفیف، کنترل موجودی
3. *Data Access (DAL)*: ارتباط با دیتابیس PostgreSQL، کش Redis
4. *Database*: ذخیره‌سازی اطلاعات

دلایل انتخاب:

- سادگی و سرعت توسعه: تیم‌های مختلف روی لایه‌های جداگانه کار کنند
- تست‌پذیری بالا: هر لایه به صورت مستقل تست می‌شود
- امنیت: لایه *Business Logic* کاملاً از *UI* جدا است
- هزینه کمتر: نیازی به معماری توزیع‌شده برای یک رستوران با ۲۰ میز نیست

تکنولوژی‌ها: *ASP.NET Core, React.js, PostgreSQL, Redis, SignalR* (برای اعلان‌های لحظه‌ای)

۷. سه خطای محتمل و روش جلوگیری

خطای ۱: ثبت دو سفارش همزمان برای یک میز

راهکار:

- استفاده از قفل خوش بینانه (*Optimistic Locking*) با نسخه گذاری (*Version Field*) در دیتابیس
- پیاده سازی *Transaction Isolation Level = SERIALIZABLE* برای جدول *Table*
- نمایش وضعیت "مشغول" برای میز به محض ثبت اولین سفارش

خطای ۲: اشتباه در محاسبه موجودی انبار (موجودی منفی)

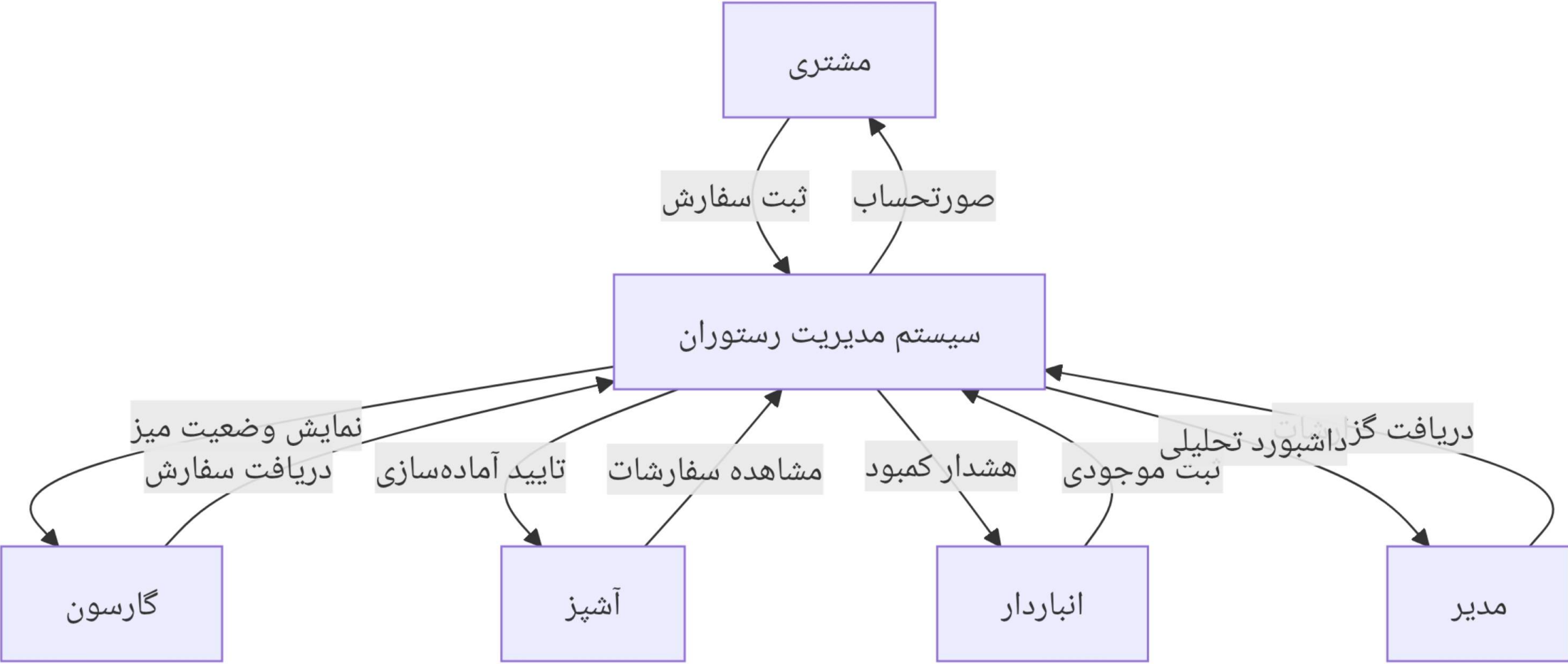
راهکار:

- استفاده از *Trigger* های دیتابیس که قبل از کاهش موجودی، موجودی فعلی را چک کند
- پیاده سازی مسیریابی جبرانی (*Compensating Transaction*) در صورت بروز خطا
- ثبت تمام تغییرات موجودی در یک جدول *Audit Log* برای بازرسی

خطای ۳: گم شدن سفارش بین آشپزخانه و گارسون

راهکار:

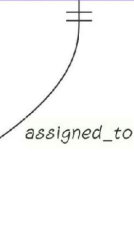
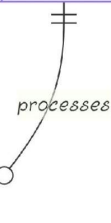
- استفاده از سیستم صف (*Message Queue*) مانند *RabbitMQ* با قابلیت *Persistent Message*
 - نمایش سفارش در دو مانیتور (آشپزخانه و ایستگاه گارسون) به صورت همزمان
 - پیاده‌سازی تاییدیه دو مرحله‌ای: آشپز تایید آماده‌سازی، گارسون تایید دریافت
 - ارسال نوتیفیکیشن از طریق *SignalR* به تبلت گارسون در لحظه
-



CUSTOMER		
int	customer_id	PK
string	name	
string	phone	
string	email	
int	loyalty_points	

EMPLOYEE		
int	employee_id	PK
string	name	
string	role	
string	shift	

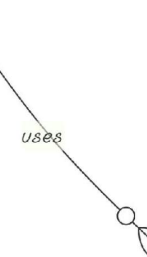
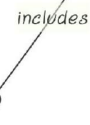
TABLE		
int	table_id	PK
int	capacity	
string	status	



ORDER		
int	order_id	PK
int	customer_id	FK
int	employee_id	FK
int	table_id	FK
datetime	order_time	
float	total_amount	
string	status	

MENU_ITEM		
int	item_id	PK
string	name	
string	category	
float	price	
string	description	

SUPPLIER		
int	supplier_id	PK
string	name	
string	phone	
string	address	



ORDER_ITEM		
int	order_item_id	PK
int	order_id	FK
int	item_id	FK
int	quantity	
float	subtotal	

INVENTORY		
int	inventory_id	PK
int	item_id	FK
int	supplier_id	FK
int	current_stock	
int	reorder_level	
datetime	last_updated	

نوع نمودار UML	کاربرد در سیستم رستوران
<i>Use Case</i>	نمایش نقش‌ها: مشتری، گارسون، آشپز، انباردار، مدیر. سناریوهای اصلی مثل ثبت سفارش، مدیریت منو، گزارش‌گیری
<i>Class Diagram</i>	طراحی کلاس‌های: <i>Order</i> , <i>MenuItem</i> , <i>Customer</i> , <i>Employee</i> , <i>Inventory</i> , <i>Table</i> با متدهای محاسبه قیمت و تایید موجودی
<i>Sequence Diagram</i>	ترتیب پیام‌ها بین گارسون، سیستم، آشپزخانه و انبار برای ثبت و آماده‌سازی سفارش
<i>Activity Diagram</i>	مدل‌سازی همزمان چند سفارش در آشپزخانه (موازی‌سازی)
<i>State Diagram</i>	مدیریت وضعیت‌های سفارش: <i>Pending</i> → <i>Cooking</i> → <i>Ready</i> → <i>Served</i> → <i>Paid</i> → <i>Closed</i>
<i>Component Diagram</i>	نمایش ماژول‌های: سفارش‌گیری، مدیریت منو، انبار، گزارش‌گیری، پرداخت

